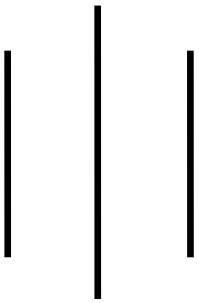
# LA GRANDEE INTERNATIONAL COLLEGE

## Simalchaur, Pokhara Nepal

A Lab Report

Of

## "Data Structure and Algorithm"



**Submitted to:**

Bachelor of Computer Application (BCA) Program

In partial fulfilment of the requirements for the degree of BCA under

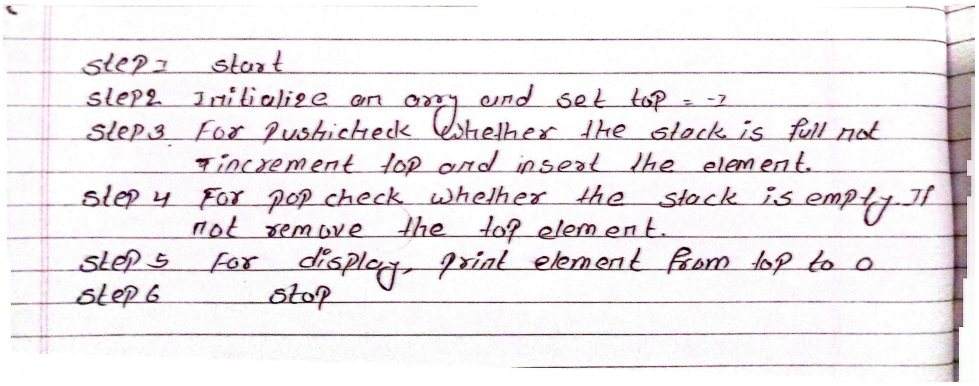
Pokhara University

**Submitted by:**


| Name: | Course | Semester | P.U. Registration Number |
|---|---|---|---|
| Binay Lamichhane | BCA | 3rd | 2025-1-53-0319 |


**Date: 05/11/2026**

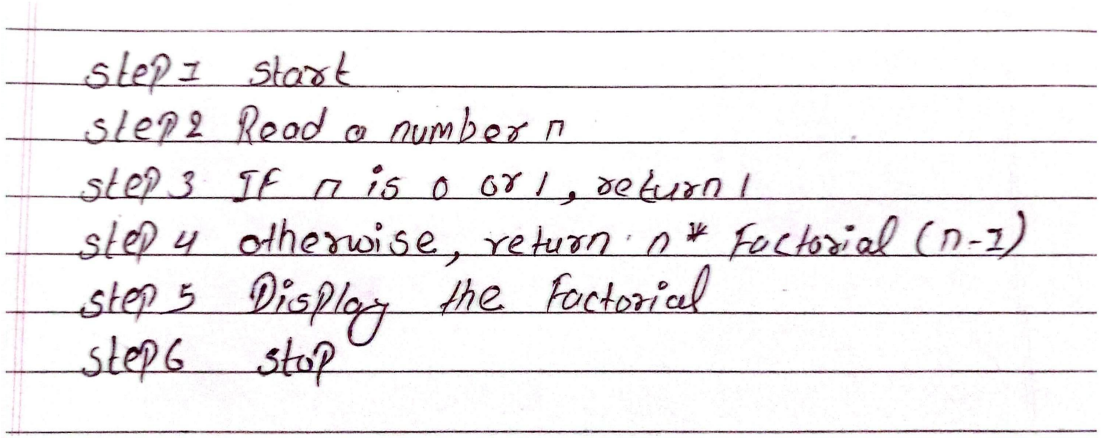
# 1. WAP to implement stack using array.




```c
#include <stdio.h>
#define MAX 5
int stack[MAX];
int top = -1;
void push(int value) {
    if (top == MAX - 1) {
        printf("Stack Overflow\n");
    } else {
        stack[++top] = value;
        printf("%d pushed\n", value);
    }
}
void pop() {
    if (top == -1) {
        printf("Stack Underflow\n");
    } else {
        printf("Popped: %d\n", stack[top--]);
    }
}
void display() {
    int i;
    if (top == -1) {
        printf("Stack is empty\n");
    } else {
        printf("Stack elements: ");
        for (i = top; i >= 0; i--) {
            printf("%d ", stack[i]);
        }
        printf("\n");
    }
}
int main() {
    push(10);
    push(20);
    push(30);
    display();
    pop();
    display();
    return 0;
}
```

```
10 pushed
20 pushed
30 pushed
Stack elements: 30 20 10
Popped: 30
Stack elements: 20 10
```

## 2. WAP using recursion to calculate the factorial of a number.




```c
#include <stdio.h>

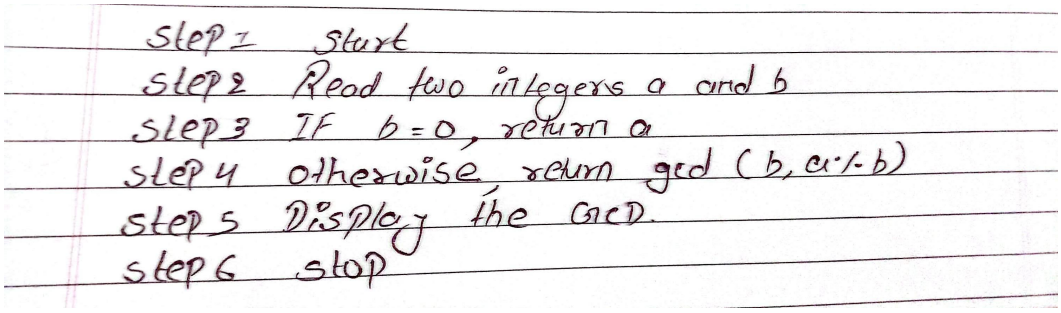
int factorial(int n) {
    if (n == 0 || n == 1) {
        return 1;
    }
    return n * factorial(n - 1);
}

int main() {
    int n;
    printf("Enter a non-negative number: ");
    scanf("%d", &n);
    printf("Factorial = %lld\n", factorial(n));
    return 0;
}
```

Output

```
Enter a non-negative number: 5
Factorial = 120
```

## 3. WAP using recursion to find the GCD of two numbers.




```c
#include <stdio.h>

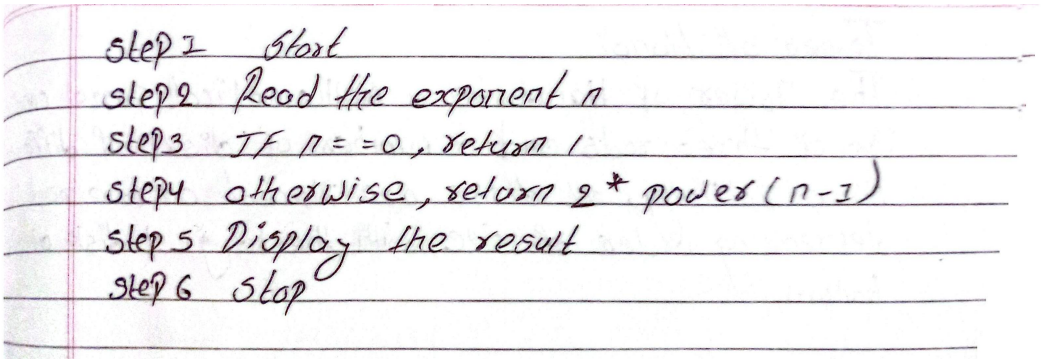
int gcd(int a, int b) {
    if (b == 0) {
        return a;
    }
    return gcd(b, a % b);
}

int main() {
    int a, b;
    printf("Enter two non-negative integers: ");
    scanf("%d %d", &a, &b);
    printf("GCD = %d\n", gcd(a, b));
    return 0;
}
```

Output

```
Enter two non-negative integers: 20 50
GCD = 10
```

## 4. WAP using recursion to find the power of 2.




```c
#include <stdio.h>

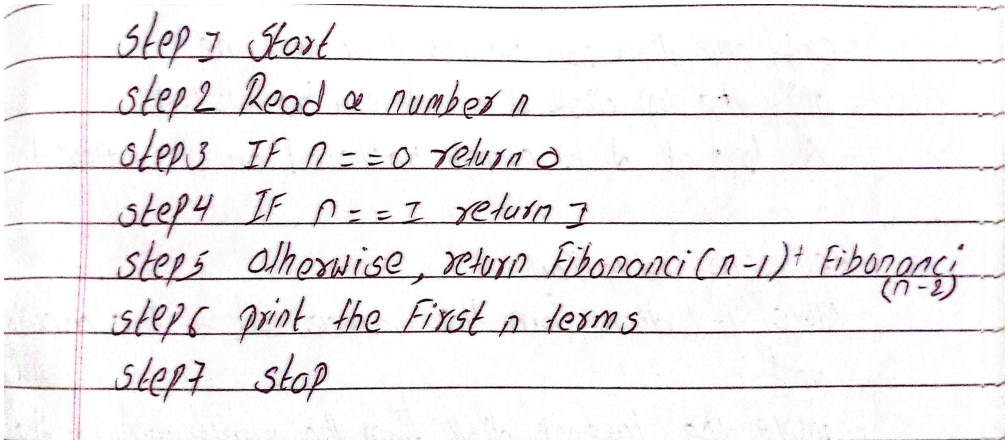
int power(int n) {
    if (n == 0) {
        return 1;
    }
    return 2 * power(n - 1);
}

int main() {
    int n;
    printf("Enter the exponent (0-62): ");
    scanf("%d", &n);
    printf("2^%d = %d\n", n, power(n));
    return 0;
}
```

Output

```
Enter the exponent (0-62): 3
2^3 = 8
```

## 5. WAP to implement Fibonacci series using recursion.




main.c

Clear Run

```c
#include <stdio.h>

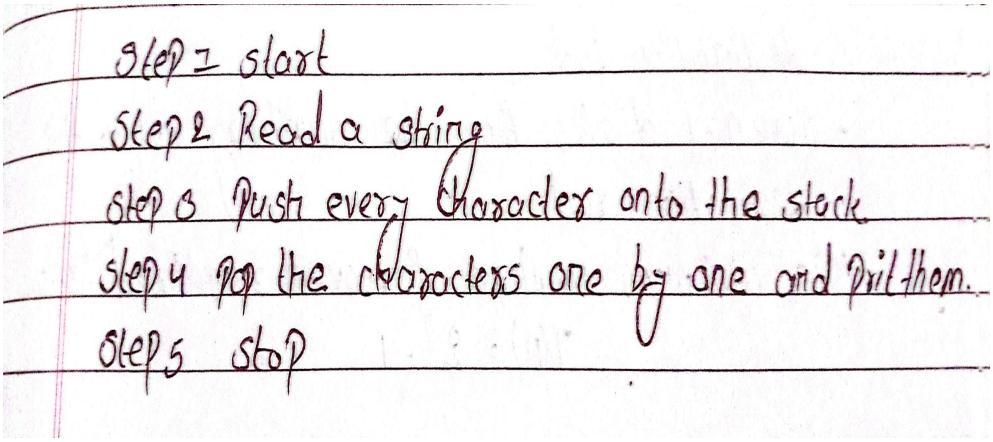
int fibonacci(int n) {
    if (n == 0) {
        return 0;
    }
    if (n == 1) {
        return 1;
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int n, i;
    printf("Enter the number ");
    scanf("%d", &n);
    printf("Fibonacci series: ");
    for (i = 0; i < n; i++) {
        printf("%d ", fibonacci(i));
    }
    printf("\n");
    return 0;
}
```

Output

```
Enter the number 10
Fibonacci series: 0 1 1 2 3 5 8 13 21 34
```

## 6. WAP to reverse a string using stack.




```c
#include <stdio.h>
#include <string.h>

#define MAX 100

void main() {
    char str[MAX], stack[MAX];
    int top = -1, i;
    printf("Enter a string: ");
    fgets(str, MAX, stdin);
    str[strcspn(str, "\n")] = '\0';
    for (i = 0; str[i] != '\0'; i++) {
        stack[++top] = str[i];
    }
    printf("Reversed string: ");
    while (top != -1) {
        printf("%c", stack[top--]);
    }
    printf("\n");
}
```

Output

```
Enter a string: Binay lamichhane
Reversed string: enahhcimal yaniB
```

**Short Notes:**

## 1. Tower of Hanoi.

Tower of Hanoi

The Tower of Hanoi is a mathematical puzzel consisting of three rods and a number of disks of different sizes. Initially, all disks are placed on one rod in decreasing order of size, with the largest disk at the bottom

Rules:

- Only one disk can be moved at a time.
- only the top disk of a rod can be moved.
- A largest disk cannot be placed on a smaller disk

Recursive solution:

- Move n-1 disk from the source rod to the auxiliary rod
- move the largest disk from the source rod in the destination rod.
- move n-1 disks from the auxiliary rod to the destination rod.

The minimum number of moves required is:

$$T(n) = 2^n - 1$$

Application: understanding recursive problem-solving and algorithm analysis

## 2. Stack ADT.

DATE

* Stack ADT :
A stack Abstract Data Type (ADT) is a linear data structure that follows the LIFO (Last In First out) principle. The element inserted last is removed first.

- Basic operations:
- Push : Inserts an element onto the top of the stack
- Pop : Removes the top elements.
- Peek / Top : Return the top element without removing it.
- is Empty : checks whether the stack is empty.
- is Full : checks whether an array-based stack is full

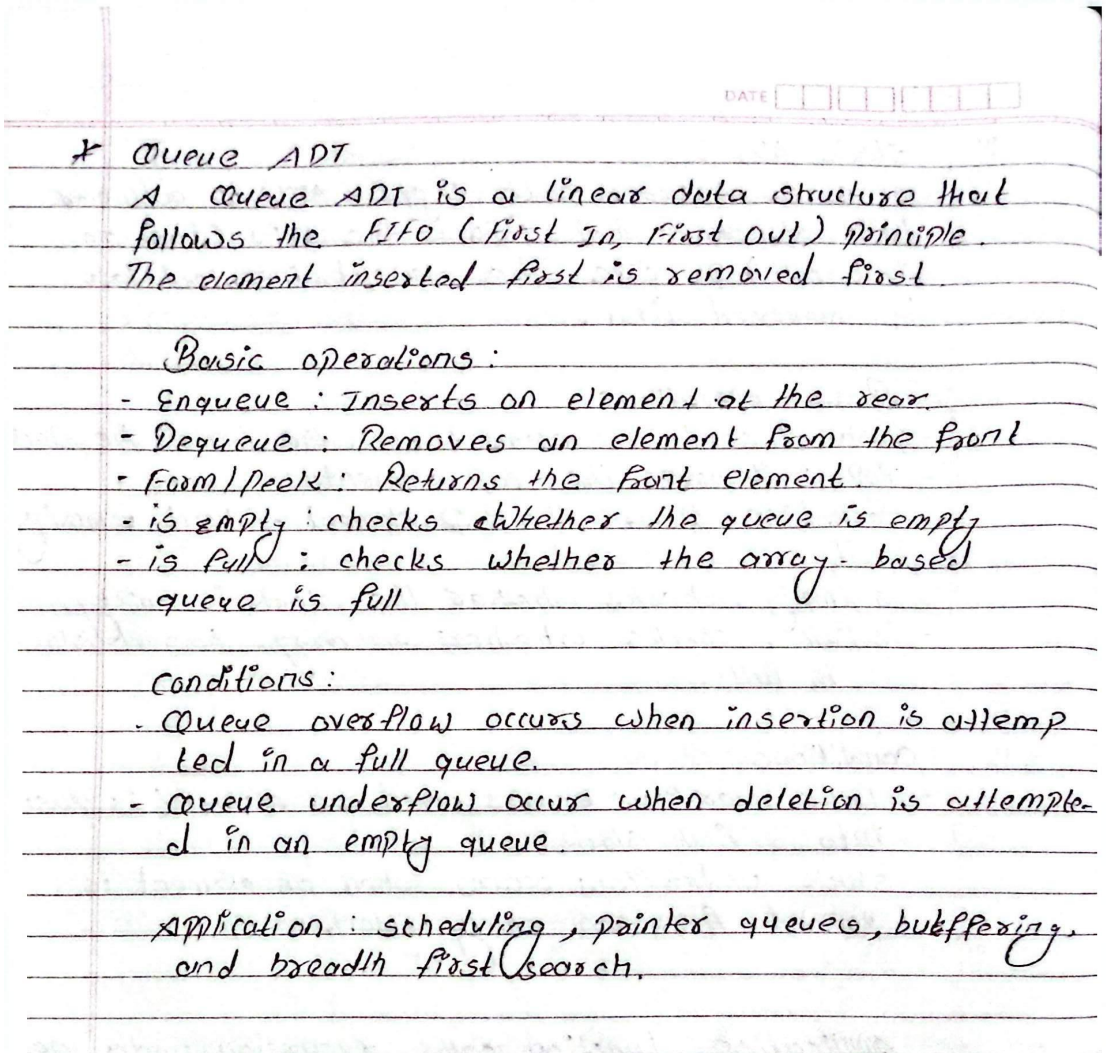
Conditions :
- Stacks overflow occurs when an element is pushed into a full stack
- stack underflow occurs when an element is popped from an empty stack.

Application : function calls, recursion, undo operation expression evaluation and parenthesis matching

classmate

PAGE

## 3. Queue ADT.



* Queue ADT
A Queue ADT is a linear data structure that follows the FIFO (First In, First out) principle. The element inserted first is removed first.

Basic operations:
- Enqueue : Inserts an element at the rear.
- Dequeue : Removes an element from the front
- Front/Peek : Returns the front element.
- isEmpty : checks whether the queue is empty
- isFull : checks whether the array-based queue is full

Conditions:
- Queue overflow occurs when insertion is attempted in a full queue.
- Queue underflow occur when deletion is attempted in an empty queue.

Application : scheduling, printer queues, buffering, and breadth first search.

**Git Hub link :** [https://github.com/binay09-ux/DSA](https://github.com/binay09-ux/DSA)